

Supplementary Material: AlgoTutorGen

Anonymous submission

Supplement Scope and Reading Guide

The main paper presents the complete argument and primary evidence. This document expands the executable interfaces, prompt contracts, formal derivations, measurement protocol, secondary tables, artifact provenance, and frozen English case studies. Its organization follows the two formal pipelines introduced in the paper: Semantic Synthesis and Verification (SSV) produces the Verified View, and Post-Verification Creative Rendering (PVCR) produces an optional Creative View.

MACHINE OK Protocol and Artifact Provenance

Decision Rule

For each page, the browser harness records nine booleans: page load, visible-answer match, reachable interaction, correct feedback, wrong feedback, hint behavior, answer-reveal behavior, learning-log behavior, and protected-answer stability. A page is MACHINE OK only if all nine are true. Correct and wrong feedback are tested using distinct submitted values. Hint and answer-reveal controls must change their associated pedagogical state, and the learning log must record the tested event. The ninth browser-audit check compares the protected visible-answer summary before and after the audited teaching actions. It is deliberately lighter than the independent noninterference stress suite, which compares algorithm-state, current-step, and artifact hashes after each action in 100 random sequences per page. Navigation in that stronger suite may change the step only to a verified target frame. These browser checks are offline evaluation instruments rather than fields of the online ReleaseGate. External requests are blocked during both audits.

Generation and Statistical Boundaries

The main comparison uses sample index 0 for each of 200 tasks, preventing tasks with more stored inputs from receiving greater weight. The full benchmark contains 646 samples. The primary ALGO TUTOR GEN condition uses DeepSeek-V4-Pro. Cross-model final-quality runs use DeepSeek-V4-Flash, GLM-5.2, and Kimi-K2.5: each starts from two candidates and two repair rounds, and failed tasks may receive a recorded retry with at most three candidates and three rounds. Consequently, the final cross-model table is not a uniform-call or

token-matched comparison. Direct HTML receives the concrete input and expected answer and is explicitly asked to generate the complete interaction suite.

McNemar tests use paired task-level binary outcomes. Paired bootstrap confidence intervals resample tasks, not frames, actions, or individual metric checks. Holm correction applies to test families, and paired ordinal or continuous secondary scores use Wilcoxon signed-rank tests where appropriate. Frame and action totals describe implementation coverage and are not treated as independent inferential samples. A nonsignificant result is not evidence of equivalence.

Formal Artifact and Release Contracts

An input is $x = (q, i, e, c)$: algorithmic problem q , concrete instance i , optional oracle e , and optional reference code or strategy c . The model produces one instrumented solver $\tilde{S}$. Executing it once under run identifier ρ yields

$$E_\rho = (y_\rho, T_\rho, Z_\rho, L_\rho),$$

where y_ρ is the final answer, $T_\rho = (e_0, \dots, e_n)$ is the ordered typed event sequence, $Z_\rho = (z_0, \dots, z_{n+1})$ is the sequence of per-step service snapshots, and $L_\rho = (\ell_0, \dots, \ell_n)$ contains runtime-captured source callsites. The SSV synthesis bundle is therefore $A = (\tilde{S}, E_\rho, G, \bar{P}, R)$, comprising the instrumented solver and execution record, SCENEGRAPH sequence G , protected pedagogical sidecar represented as the contract-checked overlay $\bar{P}$, and browser runtime R . Deterministic export produces the Verified View W_V . The optional PVCR pipeline consumes W_V and a presentation digest $D(A, x)$ and returns scoped assets H . If the creative contract $C_C(H, W_V)$ passes, the fixed shell exports a Creative View W_C ; otherwise it returns W_V .

The online release contract is

$$\Phi_{\text{release}}(A, W, x) = \Phi_{\text{execution}} \wedge \Phi_{\text{result}} \wedge \Phi_{\text{trace}} \wedge \Phi_{\text{scene}} \wedge \Phi_{\text{runtime}}, \quad (1)$$

where the terms check same-execution binding and callsites, available-oracle agreement, the typed event sequence and replayed service state, scene bindings, and runtime exportability. Offline evaluation additionally measures

$$\Phi_{\text{audit}} = \Phi_{\text{release}} \wedge \Phi_{\text{feedback}} \wedge \Phi_{\text{noninterference}}.$$

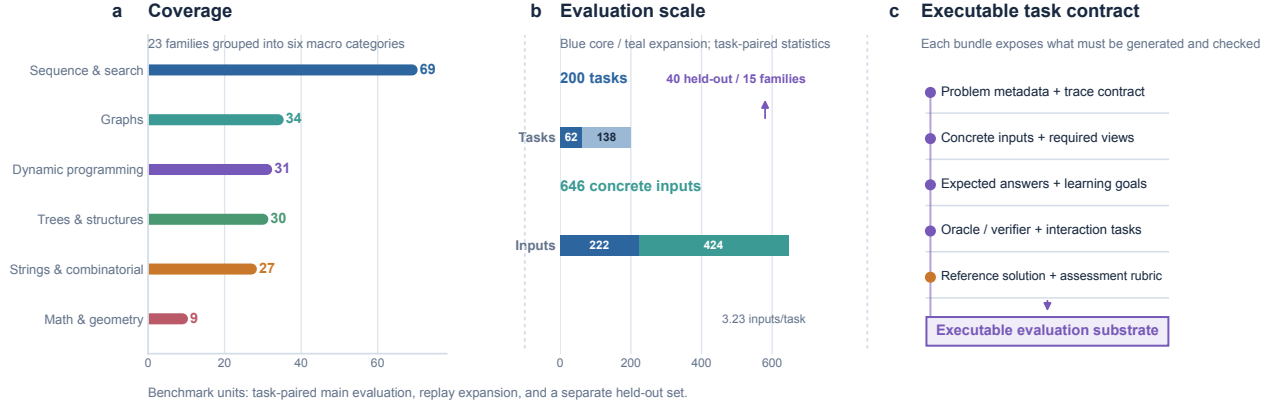


Figure S1: Complete ALGOtutorGen-Bench overview. The 200 tasks contribute one anchor input each to task-paired main comparisons and 446 additional inputs for replay-based evaluation, yielding 646 stored inputs across 23 families. Family-core contributes 62 tasks/222 inputs and expansion contributes 138/424. A separate held-out evaluation contains 40 new tasks from 15 supported families.

The feedback term covers distinguishable correct and incorrect responses, hints, answer reveal, and learning-log behavior. The noninterference term requires teaching actions to preserve algorithm facts, while navigation may select another verified frame. These stronger checks are evaluation instruments rather than fields of the online ReleaseGate.

Direct HTML searches one open-ended artifact containing answer data, algorithm state, rendering logic, handlers, and teaching state. ALGOtutorGen instead implements the refinement chain

$$\begin{aligned} \tilde{S} &\xrightarrow{C_E} E_\rho \xrightarrow{C_T} G \xrightarrow{C_G} R, \\ (T_\rho, G) &\xrightarrow{\text{teaching}} P \xrightarrow{C_P} \bar{P}, \\ (G, R, \bar{P}) &\xrightarrow{C_B} W_V. \end{aligned}$$

The operational benefit comes from early rejection, reuse of verified semantic data, deterministic downstream stages, and narrower failure attribution.

Executable Interfaces and Configuration

Table S1 states the implementation boundary of every component. The optional correctness-contract drafter remains distinct from the instrumentation layer: instrumentation binds an execution to its trace, whereas the oracle tests whether the executed algorithm is correct. The principal path begins with one instrumented solver.

SSV ends at the release of the Verified View. PVCr is an optional downstream presentation pipeline: it consumes a verified artifact, operates inside the fixed tutor shell, and is omitted from the preservation and noninterference premises. Its accepted output is called the Creative View; any creative-generation, sanitizer, browser, or layout failure returns the Verified View.

JSON generation uses temperature 0.2 and text generation uses 0.3, except Kimi models, for which the provider-required value is 1.0. The API client allows four JSON-parse retries and two transport retries by default; these calls are included

in logged model-call metadata when present. Remote model versions can drift.

Sandbox and Threat Model

Generated instrumented solvers and optional verifier functions are parsed with an AST check, executed in a separate Python child process, and receive a restricted builtin namespace plus allowlisted standard-library modules. Each function has a 30-second execution timeout, and a trace event whose serialized state exceeds 20,000 characters is rejected. The instrumentation layer binds each service event to the executing frame or AST source map and assigns the run identifier; these fields are not accepted from model output. The DSL guard rejects undocumented object methods before execution and protects service-owned visible state from direct mutation. This boundary is intended to contain accidental model-generated code and enforce the artifact contract; it is not an operating-system or container security claim against a determined adversary.

Service-Compositional Trace Interface Catalog and Emission Semantics

The TraceSession catalog contains 21 factories. They are grouped below by the state role they expose to the trace program; grouping is descriptive and does not determine which algorithm families may use a service.

A supported service operation has two atomic effects:

$$(z_t, o_t) \mapsto (z_{t+1}, e_t).$$

It updates the represented service state and emits a typed event with stable object references, before/after values, run identifier, and automatically captured callsite. For example, `arr[i] = value`, `heap.push(x)`, `ptr.move(j)`, and `dist[node] = d` emit the corresponding set, container, pointer, or map event. Ordinary Python control flow and local variables may compute temporary values, but any canonical fact claimed by the released tutor must be introduced through a service or explicit checked semantic event.

Component	Input	Output	Construction	Blocking boundary and failure handling
Instrumented-solver generation	Problem, concrete input, optional expected result, requested variants	One service-compositional solver per variant	Generated	Strict schema and function signatures; no independently authored <code>solve/trace</code> pair
Single execution and materialization	Instrumented solver and concrete input	E_ρ : answer, events, per-step service snapshots, and source callsites	Sandboxed execution	Common run ID; callsite and source-range checks; prefix/final replay equality; oracle agreement reported separately
Scene compilation	Validated SEMANTIC-TRACE	SCENEGRAPH frames	Deterministic	One event maps to one frame; structural scene gate checks nonempty/visible objects and references
Teaching enrichment	Validated trace-event digest and compiled SCENEGRAPH	Teaching/interaction overlay applied to the compiled scene	Generated	Strict overlay schema and sanitization; primary 30-event context retries with three events; failure remains a warning
SSV export and runtime	Artifact, scene, sanitized teaching	Verified View and side-car JSON	Deterministic	Online structural release gate; external-resource, nine-check browser, and noninterference tests are reported offline
PVCR	Verified artifact/frame context	Scoped stage CSS/template/renderer	Generated	Presentation sanitizer plus browser/layout gate; rejection falls back to the Verified View; supplied nested context is not a deep-frozen security boundary

Table S1: Executable component interfaces. “Generated” denotes an open-ended model output; “deterministic” denotes project code operating on validated data.

The final answer is committed by this same execution. The DSL guard statically tracks variables constructed by these factories and rejects undocumented methods and visible-state bypasses before or during sandbox execution.

Full-200 Execution Trace Audit

The confirmatory audit evaluates the primary selected-final system. Its unit of evidence is (task, variant, input, ρ), not merely a task. All 200 tasks retain two accepted variants, giving 400 variant programs. Running both variants on every stored task input gives $2 \times 646 = 1,292$ instrumented executions. Event- and prefix-sensitive checks are aggregated strictly at the run level: a run passes only if every applicable event or replay prefix passes.

Same-execution binding is established at record construction: the runtime issues ρ and attaches it to the answer commitment, events, and snapshots. The audit does not accept a post-hoc pass that merely copies one string into otherwise independent outputs. Callsite validation checks both source range and the frozen source hash; a forged but in-range line therefore remains invalid. Prefix replay starts from the independently stored initial service state and applies event transitions without executing the generated solver. This establishes logging consistency for service-mediated state. Oracle agreement independently evaluates whether the traced answer matches the expected result.

The visible-state closure audit combines the service ownership rule with end-of-step canonical digests. Ordinary local variables are allowed and are outside the claim. A value be-

comes protected when it is exported as a canonical page fact; direct mutation of such state without a corresponding service operation is rejected. Deterministic replay is performed in a fresh process with fixed source and input hashes, random seed, and environment manifest. Equality is computed over canonical answer, event, and state digests rather than run IDs or timestamps.

All eight prespecified checks pass. The result establishes that the accepted answer, event sequence, per-step service snapshots, runtime-captured source callsites, and replayed states are mutually bound to one instrumented execution for every tested run. Oracle agreement is reported separately from replay equality because it evaluates the final algorithmic result, whereas the other rows evaluate the integrity of the execution record.

Negative Fault Injection

For each accepted run, the injector selects one applicable site for each fault family: deleting a state-changing event, swapping an order-sensitive adjacent pair, changing a before/after value, changing a canonical snapshot, changing the final answer, forging a callsite, or bypassing a service to mutate protected visible state. Each mutation is revalidated from a fresh copy of the accepted record. This produces 1,292 injected cases per family and 9,044 negative cases in total. Deletions of explanation-only events and swaps of commutative operations are not selected, because they need not change canonical semantics. One semantics-preserving reserialization control is also evaluated for every accepted

Condition	Model	Candidate/repair policy	Runtime limits	Important boundary
ALGOTUTORGEN main	DeepSeek-V4-Pro	2 solution variants; 2 candidates; 2 repair rounds per candidate	32,768 output tokens; 900 s API timeout; 3,000 s task timeout; concurrency 8	Strict warnings; teaching enabled; sample index 0
Direct HTML main	DeepSeek-V4-Pro	1 candidate; at most 2 repairs	32,768 output tokens; 2,400 s task timeout; concurrency 8	Expected result visible; no browser-feedback repair; self-contained HTML requested
Cross-model ALGOTUTORGEN	Flash, GLM, Kimi	Initial 2×2 ; failed tasks eligible for final-quality retry up to 3×3	Browser audit sharded 16 ways	Final frozen pages reported; calls/tokens differ from Direct
Held-out ALGOTUTORGEN	DeepSeek-V4-Pro	2 candidates; 2 repair rounds	900 s API timeout; 2,400 s task timeout; concurrency 8	Forty new tasks from 15 families
WebGen-Agent	DeepSeek-V4-Pro	At most 5 agent iterations	Same final browser audit	Task-adapted external code path; not a reproduction of its original benchmark
HTMLCURE (STRICT)	DeepSeek repair; Gemini evaluator	1 repair iteration	External requests blocked in final audit	Operates on frozen Direct pages; self-containment is part of strict failure
DIRECT-BROWSERREPAIR (MAX 5)	DeepSeek-V4-Pro	Adaptive maximum budgets 0/1/2/3/5; early stop; best-so-far	32,768 output tokens; 1,800 s API timeout; concurrency 32	Same 200 hash-verified initial pages; full previous HTML; generic browser feedback

Table S2: Recorded generation and execution configuration. Counts are maximum policy budgets, not a claim that every task consumed the maximum.

State role	Service factories
Indexed and tabular	array, string, table, intervals, points
Scalar and associative	scalar, map, counter, pointer
Ordered containers	heap, stack, queue, deque
Linked and hierarchical	linked_list, trie, tree, union_find
Graph and range structures	graph, flow_network, fenwick, segment_tree

Table S3: The current service catalog. The generation interface exposes service capabilities rather than a selector over algorithm-family templates.

run.

All case-level and aggregate outputs are stored in `single_execution_trace_audit.json`. The top-level record includes `schema_version`, source commit and source hash, environment manifest, and scope counts for tasks, variants, inputs, runs, and events. Each case stores stable task, variant, and input identifiers, ρ , source hash, answer and state digests, event count, the eight check outcomes, oracle details, and mutation outcomes. The aggregate section reports numerators and denominators rather than only rounded percentages.

Every applicable corruption in the seven fault families is rejected, while all semantics-preserving controls are accepted. This paired positive/negative design rules out a validator that passes untouched records without checking them or rejects every modified record indiscriminately. Together with Table S5, the result establishes mechanical fidelity to the instrumented execution.

Wrong-but-Self-Consistent Solver Audit

Beyond record-level corruptions, we mutate solvers before execution to construct 60 wrong-but-self-consistent runs across 30 tasks spanning all 23 algorithm families. All 60 pass internal replay-consistency checks, yet the oracle-backed pipeline detects every result mismatch and the release gate rejects every case, yielding zero erroneous releases and demonstrating independent semantic assurance before release.

Blinded Dual-Reviewer Source-to-Trace Audit

We therefore evaluate algorithmic validity separately. Two reviewers from the research team with algorithms and data-structures training label 120 complete traces from 60 stratified tasks spanning all 23 families. Both generated variants are audited for every sampled task, yielding 3,883 event judgments. The blinded review interface shows the problem, concrete input, expected output, complete source with stable

Case	Observed service composition	Family-independent state roles
Coin change	array	Denominations and evolving values
Dijkstra shortest path	graph + heap + map	Topology, frontier priority, tentative distances
Permutations	array + tree	Candidate sequence and recursive call structure
Number of provinces	array + graph + scalar + table + tree + union_find	Input matrix, components, counters, recursive and disjoint-set views

Table S4: Representative service compositions recovered from the saved qualitative program source.

Check	Full-200 result	Interpretation
Same-execution binding	1,292/1,292 runs	Answer, events, and snapshots share one runtime-issued ρ
Runtime callsite coverage	1,292/1,292 runs	Every event in every passing run carries a non-model-authored runtime callsite
Source-range validity	1,292/1,292 runs	Every callsite in every passing run resolves under the frozen source hash
Prefix replay equality	1,292/1,292 runs	Every independently replayed prefix matches its recorded service state
Final-state equality	1,292/1,292 runs	Replay-final and execution-final state digests agree
Oracle agreement	1,292/1,292 runs	Instrumented answers agree with the trusted expected results
Unlogged visible mutation	0/1,292 runs	No protected canonical fact changes without a service event
Deterministic replay	1,292/1,292 runs	Fresh-process canonical answer, event, and state digests agree

Table S5: Observed results of the Full-200 execution audit over 200 tasks, 400 accepted variant programs, and 1,292 instrumented executions obtained by running two variants on each of 646 stored task inputs. Event- and prefix-sensitive rows count a run as passing only when all of its events or prefixes pass.

line numbers, ordered events, source references, pre-state, operation, post-state, explanation, and final output. Method identity, automatic gate outcomes, and the other reviewer’s labels are hidden. Original labels produced independently under this blinding are retained for agreement statistics; disagreements and disputed algorithmic interpretations receive a separate technical review.

The audit finds no confirmed critical semantic error. A disputed dynamic-programming event initially flagged as an incorrect transition is confirmed by technical review to match the source and recurrence, and is corrected before computing the final outcome. The one remaining non-perfect task contains a single noncritical explanation inconsistency in one variant under the final labels. Among the 3,880 source-verifiable events, two point to non-adjacent lines in the same logical statement block and remain semantically aligned; no event has an incorrect source attribution. The three source-unverifiable events are excluded from the source-alignment denominator and do not by themselves fail trace-perfect status. No reviewed event contains an incorrect state transition or algorithmic operation.

The human-audit gates specified before labeling require a task-perfect Wilson lower bound of at least 90%, a critical-error one-sided upper bound of at most 5%, a source-alignment lower bound of at least 90%, and Cohen’s $\kappa \geq 0.60$ for the key judgments. The observed values are 91.1%, 4.9%, 99.95%, and 1.000, respectively. All four gates pass, and the outcome specified before labeling is Pass. We therefore report high sampled algorithmic trace validity and a low critical-error bound.

Injected fault	Expected detection path	Observed outcome
Delete state-changing event	Prefix or final replay mismatch	1,292/1,292 rejected
Swap order-sensitive events	Prefix replay, dependency, or before-state mismatch	1,292/1,292 rejected
Modify before/after	Atomic-transition validation or replay mismatch	1,292/1,292 rejected
Modify canonical state	Snapshot/replay digest mismatch	1,292/1,292 rejected
Modify final answer	Same-run answer commitment, final event, or oracle mismatch	1,292/1,292 rejected
Forge source line	Runtime callsite/source-hash binding failure	1,292/1,292 rejected
Bypass a service mutation	Visible-state closure or end-of-step digest mismatch	1,292/1,292 rejected
All negative injections	Any applicable validator path above	9,044/9,044 rejected
Semantics-preserving control	Canonical answer, event, and state digests remain equal	1,292/1,292 accepted

Table S6: Negative fault-injection results for the execution-record validator. One applicable mutation from each family is injected into every accepted run. A paired semantics-preserving control rules out a trivial reject-all validator.

Human-audit measure	Result	Interval or interpretation
Trace-perfect task	59/60 (98.3%)	Wilson 95% CI [91.1, 99.7]
Critical-error task	0/60 (0.0%)	One-sided 95% upper bound 4.9%
Trace-perfect variant	119/120 (99.2%)	Descriptive variant-level result
Critical-error variant	0/120 (0.0%)	No confirmed critical semantic error
Source exact or adjacent	3,878/3,880 (99.95%)	Strict line-proximity measure
Source non-adjacent but aligned	2/3,880 (0.05%)	Same logical statement block
Incorrect source attribution	0/3,880 (0.0%)	No confirmed source-attribution error
Source unverifiable	3/3,883 (0.08%)	Excluded from the source-alignment denominator
State transition correct	3,883/3,883 (100.0%)	Final technically reviewed labels
Algorithmic operation correct	3,883/3,883 (100.0%)	Final technically reviewed labels
Explanation consistent	3,882/3,883 (99.97%)	One noncritical inconsistency in one variant
Trace-perfect agreement	Cohen’s $\kappa = 1.000$	Gate specified before labeling passed
Critical-error agreement	Cohen’s $\kappa = 1.000$	Gate specified before labeling passed

Table S7: Blinded dual-reviewer source-to-trace audit of source alignment, state-transition and operation correctness, and explanation consistency. The task is the primary statistical unit; a task is trace-perfect only when both generated variants are trace-perfect. Agreement uses the original independently produced labels, whereas correctness results use the final technically reviewed labels.

Full-200 Atomic-Service and Separated-Trace Ablation

To isolate atomic state/event ownership, we run a separate paired Full-200 ablation under the same tasks, DeepSeek-V4-Pro model, generation policy, validators, compiler, and fixed shell. Atomic-service uses `TRACESESSION` operations that jointly mutate represented state and emit events. The ALGOGEN-style separated-trace condition maintains algorithm state in ordinary Python variables, requires the model to emit the corresponding event claim separately after a state change, and provides no helper that binds the two operations. It retains the same downstream pipeline and therefore serves as an interface-level matched control rather than a reimplementa-tion of ALGOGEN’s VTA/RSL renderer.

The MACHINE OK discordant pairs are 36 Atomic-service-only and 15 Separated-trace-only; generation-pass discordances are 35 and 12. After excluding the 23 pilot cases, the remaining 177 pairs retain the same direction: MACHINE OK is 164/177 versus 142/177 (+12.43 points; 95% CI [5.65, 19.77]; McNemar $p = .00126$), and generation pass is 168/177 versus 144/177 (+13.56 points; 95% CI [6.78, 20.34]; $p = .000182$).

Atomic-service and Separated-trace produce auditable ex-

ecution records for 187 and 164 tasks, respectively, with 152 paired records observable in both conditions. All 152 pairs pass execution validation, same-execution binding, prefix replay, absence of unlogged mutations, and state/event agreement in both arms. Atomic ownership therefore raises the yield of valid auditable records while preserving the same validation standard. Observed Separated-trace generation failures include nine claim mismatches, five missing pending state transitions, and one unlogged transition, matching the coordination burden removed by atomic service operations.

Measure	Atomic-service	Separated-trace	Difference	95% CI	Paired test
MACHINE OK	183/200	162/200	+10.50 pp	[3.50, 17.00] pp	McNemar $p = .00460$
Generation pass	187/200	164/200	+11.50 pp	[5.00, 18.00] pp	McNemar $p = .00109$
Model calls/task	6.235	7.025	-0.790	[-1.540, -0.025]	-
Repair calls/task	1.000	1.880	-0.880	[-1.130, -0.630]	-
Tokens/task	85,189.8	94,081.2	-8,891.3	[-19,072.0, 1,526.9]	-
API latency/task	615.7 s	698.6 s	-82.8 s	[-155.9, -8.5] s	-
End-to-end time/task	619.6 s	704.1 s	-84.5 s	[-158.0, -9.8] s	-

Table S8: Paired Full-200 atomic-service ablation. The separated-trace condition is an ALGOGEN-style state/event interface under the same downstream tutor stack. Differences are Atomic-service minus Separated-trace. Confidence intervals use 10,000 task-level paired-bootstrap resamples; binary outcomes additionally use exact two-sided McNemar tests.

Full-200 Service Reuse and Prompt-Guidance Ablation

We evaluate service reuse on a separate frozen Full-200 artifact set rather than pooling it with the 400-variant, 1,292-run execution audit. The service audit contains 200 tasks, 399 accepted variants, and 23 algorithm families; one task retains only one accepted variant. Twenty of the 21 catalog services appear, `intervals` is unused, and no out-of-catalog `TraceSession` factory call is observed. At least two services are composed in 187/200 tasks (93.5%) and 297/399 variants (74.44%). Fifteen services recur across at least two algorithm families, and 13 recur across at least two broader analysis groups. The most widely reused service, `array`, appears in 138 tasks and 18 families.

The paired prompt ablation uses the same 200 tasks, inputs, DeepSeek-V4-Pro model, temperature, candidate and repair budgets, token limit, timeout, and exposed oracle. The hybrid condition retains the service API, output schema, family templates, and task strategy guidance. The service-only condition preserves the service API, schema, and neutral composition examples but removes the family templates and strategy guidance. Differences below are Service-only minus Hybrid; bootstrap intervals use 10,000 task-level resamples. All paired results use frozen Full-200 outcomes under the same Hybrid and Service-only conditions.

The results support broad service reuse while showing that the current family and strategy guidance remains operationally useful.

Relation to ALGOGEN’s VTA and RSL

ALGOGEN and ALGOTUTORGEN share the premise that execution state should be separated from deterministic rendering, but they use different generation abstractions and target different released artifacts (Authors et al. 2026). ALGOGEN’s VTA-JSON defines a shared many-sorted operation core; its tracker instructions additionally specify graph-, dynamic-programming-, sorting-, and other family requirements, and RSL generation consumes algorithm-family context. ALGOTUTORGEN moves the generation boundary to service operations and adds a fixed interactive-teaching shell. The Full-200 separated-trace arm imports this shared state/event separation into the same tutor compiler and shell, making the interface contribution experimentally identifiable without conflating released modalities. Table S10 states this

distinction without treating the two systems as directly comparable on one performance number.

Prompt Engineering

Table S11 presents the prompts by role before giving representative instructions, following a module-first organization. Dynamic user messages are assembled from the concrete problem, input, expected output when exposed by the protocol, verified execution-record digest, or a structured failure report. The anonymous artifact must bundle the exact instrumented-solver and repair prompts used for the confirmatory run. Some production prompts are Chinese; the English boxes below preserve their operational structure and constraints but are not byte-identical transcripts.

Representative English Prompt Skeletons

System Prompt: SSV Executable Specification

Role: Generate one executable instrumented solver per requested variant. Do not generate independent `solve(input_data)` and `trace(input_data)` functions.

Return: One strict JSON object containing `problem_title`, `input_contract`, optional `verifier_code`, and one or more variants.

Execution contract: Use ordinary Python control flow over documented `TraceSession` factories and methods. The same execution must compute the final answer and produce all visible per-step service snapshots and events.

Checks: Every canonical page fact must originate from service state or an explicit checked event; every value must be JSON serializable; the runtime supplies the run ID and callsite.

Forbidden: A second answer-producing function; model-authored `code_line`; direct writes to protected visible state; HTML, CSS, JavaScript, `renderer` fields, project imports, undocumented DSL methods, or extra schema keys.

Measure	Hybrid	Service-only	Difference (pp)	95% CI / decision
First specification valid	121/200	85/200	−18.0	[−27.0, −9.0]; Holm $p = .000471$
Final generation pass	200/200	181/200	−9.5	[−14.0, −5.5]; Holm $p = .0000572$
MACHINE OK	198/200	181/200	−8.5	[−12.5, −4.5]; noninferiority failed
Unknown-DSL-call-free	194/200	171/200	−11.5	[−17.0, −6.5]; Holm $p = .000238$

Table S9: Prompt-matched Full-200 ablation. Service-only produces 181 MACHINE OK tutors versus 198 for Hybrid and does not meet the prespecified −3-percentage-point noninferiority boundary.

Dimension	ALGOGEN	ALGO TUTOR GEN
Released artifact	Algorithm visualization rendered to video, figure, or interactive backend	Self-contained interactive tutor with trace, controls, feedback, hints, answer reveal, and learning log
Generated trace interface	Executable tracker emits VTA-JSON operations	Python control flow composes <code>TraceSession</code> service objects that emit <code>SEMANTICTRACE</code> events
Conditioning unit	Shared VTA core plus view-specific and algorithm-family guidelines	Service capabilities and their composition; no family label enters the service factory interface
State/visual update	Algorithm-state update and VTA visualization operation are separately authored	Supported service mutation and semantic-event emission are coupled for represented state
Rendering control	Deterministic renderer controlled by RSL, including family context	Deterministic <code>SEMANTICTRACE-SCENEGRAPH</code> -browser path; optional compiler hints remain outside the generation interface
Pedagogy and interaction	Not the central artifact contract	Fixed shell owns code, timeline, teaching, interaction, feedback, log, and answer
Creative presentation	RSL specializes style before backend rendering	Optional PVCR runs after semantic release and owns only the central stage slot
Failure boundary	Tracker, VTA schema/semantics, RSL, and backend rendering	Specification, service/trace, scene, teaching, runtime, and optional presentation boundaries
Scope limitation	Family-conditioned guidance remains part of the published workflow	The current prompt retains high-frequency family recipes and one benchmark-specific clarification; compiler hints also remain, so only the generated service interface is claimed to be family-agnostic

Table S10: Design-space comparison between ALGOGEN and ALGO TUTOR GEN. The target modalities differ, so the table describes interfaces and ownership rather than asserting a direct performance ranking.

System Prompt: SSV Targeted Repair

Context: Original request, previous complete specification, and typed schema, sandbox, answer-agreement, state-size, or release diagnostics.

Instruction: Return a complete replacement JSON object, but make the smallest correction that resolves the reported boundary. Preserve variant count, function signatures, and DSL style.

Failure localization: Schema errors change structure; Python exceptions change the indicated code path; replay mismatches repair the corresponding service-mediated transition; oracle disagreement repairs the algorithm rather than rewriting trace metadata; oversized events retain only visualization-relevant state.

Forbidden: Splitting the solution into independent solver and tracer functions, manually filling callsites, bypassing services for visible state, or inventing object methods.

System Prompt: Read-Only Teaching Enrichment

Source of truth: A digest of selected, validated `SEMANTICTRACE` frames, including operation, targets, dependencies, before/after values, state diff, adjacent-step summaries, and code line.

Return: `frames[step, teaching, interaction]` only. Teaching may contain what, why, formula, invariant, common mistake, and hint; interaction may contain one grounded prediction checkpoint.

Grounding rule: The answer and feedback must follow from the supplied frame.

Forbidden: Changing operation, targets, dependencies, state, result, event order, or code line.

Module	Context	Required output	Hard boundary	Retry trigger
Executable spec	Problem, input, optional expected, requested variants	Strict instrumented-solver JSON	Fixed fields; one executable per variant; documented <code>TraceSession</code> DSL; no model-authored run ID/callsite; no HTML/rendering	JSON parse, schema, sandbox, execution-record, replay, oracle, or release failure
Optional correctness contract	Problem, input, optional expected	<code>correctness-contract-v1</code> JSON	Expected has priority; oracle returns answer and is not a proof	Schema/oracle failure
Spec repair	Previous complete JSON plus structured diagnostics	Complete replacement instrumented-solver JSON	Minimal targeted change; preserve signatures and DSL; no invented methods or manually supplied call-sites	Bounded round while candidate remains
Teaching overlay	Digest of verified frames	step, teaching, interaction only	Read-only algorithm facts; check-point answer must follow from supplied frame	Parse/schema failure; retry with three frames
PVCR generation	Verified artifact and current frame context	Scoped style/template/render function	Presentation-only generation contract; sanitizer checks required scripts and stable IDs	Browser/layout failure; fallback to Verified View
PVCR repair	Previous creative assets, verified digest, structured overlap/clipping report	Replacement scoped creative assets	Repair presentation only; preserve shell IDs, trace, state, and result	One bounded repair before Verified View fallback
Direct HTML baseline	Same task/input/expected and teaching requirements	Complete self-contained HTML	No private contract claim; all page behavior generated together	Whole-page structured repair
Screenshot judge	Screenshot and rubric	Strict score JSON	Visible evidence only; no correctness or release decision	Parse retry only

Table S11: Prompt modules and hard boundaries.

System Prompt: Post-Verification Creative Rendering

Read-only context: Problem description, concrete input, verified result, release summary, state-key summary, and selected verified frames.

Return: Only `<style id="creative-stage-style">`, an optional stage template, and a script defining `window.renderCreativeStage(ctx)`.

Scope: Draw the problem-specific main stage inside the deterministic shell; all frame changes must read `ctx.frame`, `ctx.state`, and `ctx.evidence`.

Forbidden: A complete page, duplicate timeline or feedback controls, network/storage access, recomputation, reserved shell IDs, or writes to artifact facts.

System Prompt: PVCR Layout Repair

Context: The previous scoped stage assets plus a structured browser report for overlap, clipping, text occlusion, or weak scenario salience.

Repair objective: Replace opaque overlays with outlines, halos, side bands, or low-opacity layers; separate data marks, labels, legends, and backgrounds; keep a stable layout skeleton across frames.

Invariant: Repair only the main-stage presentation. Do not change the verified input, trace, state, result, shell panels, or controls.

Output: A complete replacement of the same three scoped PVCR assets.

System Prompt: Direct HTML Baseline

Input: The same problem, concrete input, strategy hint, expected result when visible under the protocol, and teaching requirements.

Return: One complete offline HTML tutor containing readable code, current state, synchronized timeline and navigation, visible algorithm objects, prediction checkpoints, distinct correct and incorrect feedback, hint, show answer, learning log, and final answer.

Boundary: The page must not claim SceneGraph, release-gate, or machine validation.

Repair: A structured failure causes whole-document replacement; there is no independent trace or scene checkpoint.

System Prompt: Screenshot Quality Judge

Evidence: One screenshot, condition label, viewport, phase, and a frozen rubric.

Score only: Layout readability, algorithm-state visibility, teaching explanation, interaction affordance, evidence alignment, and overall teaching quality on integer 1–5 scales.

Boundary: Use visible evidence only. Do not read source code, infer hidden runtime state, judge answer correctness, or make a release decision.

Return: Strict JSON with scores, confidence, visible issues, and one short caption suggestion.

Post-Verification Creative Rendering

PVCR is a downstream synthesis pipeline rather than a continuation of semantic generation. Its purpose is to recover

problem-specific visual communication after SSV has already fixed the algorithm facts, interaction structure, teaching controls, and browser release. This ordering allows the creative renderer to use open-ended CSS, template structure, and JavaScript drawing logic without asking it to recompute the algorithm or reconstruct the entire tutor page.

Verified Context Construction

For an accepted SSV artifact A and input x , the context builder constructs a bounded presentation digest

$$D(A, x) = (q, i, r, g, k, \tau, F),$$

where q is the compact problem description, i is the concrete input, r is the verified result, g is the release-gate summary, k summarizes state keys and object types, τ summarizes trace length and operations, and F contains selected representative frames. Frame selection preserves the current state, evidence, code reference, and neighboring-step summaries needed to design a stable visual grammar. Large raw traces are not copied verbatim into the prompt; the model receives enough verified structure to specialize the scene without becoming a second execution engine.

The same verified context is available at runtime through `ctx`. Its public fields include the creative-stage host, verified artifact, active variant, compiled scene, current frame, frame sequence, frame index, concrete input, verified result, current state, evidence, and fixed escaping/formatting helpers. The runtime calls the renderer again whenever the verified timeline changes. Consequently, animation and navigation remain driven by the existing frame sequence rather than by model-generated state transitions.

Scoped Asset Contract and Fixed-Shell Integration

The creative model returns three assets and no complete webpage:

1. `<style id="creative-stage-style">` containing styles scoped to the creative stage;
2. an optional `<template id="creative-stage-template">` containing reusable stage structure; and
3. a script defining `window.renderCreativeStage(ctx)`.

The deterministic exporter embeds these assets into the already released tutor shell. The shell continues to own the problem statement, source code, timeline, step navigation, evidence, prediction checkpoint, correct/incorrect feedback, hint, answer reveal, learning log, debug panels, and verified answer. PVCr owns only the problem-specific main-stage presentation.

This interface is intentionally asymmetric. The creative renderer can read verified frame context and draw into `ctx.host`, but it is prohibited from redefining the trace, result, active step, variants, or teaching controls. Stable shell identifiers are reserved, and the template extractor removes page-level `html`, `head`, `body`, `doctype`, nested script, and wrapper-template tags before integration. The runtime

freezes the top-level context object and reinvokes the renderer through a fixed entry point. The nested artifact context is not presented as a general-purpose security sandbox; the combined generation, embedding, resource, and browser boundaries define the accepted threat model.

Sanitization, Browser Gates, Repair, and Fallback

Acceptance is cumulative. First, the generation contract prohibits full-page output, duplicate shell controls, external libraries, network access, persistent storage, dynamic code loading, recomputation of algorithm facts, and writes to verified state. Second, the asset sanitizer requires an embeddable script, the `renderCreativeStage` entry point, and absence of reserved shell identifiers in the template. Third, the fixed exporter integrates only the extracted scoped assets. Fourth, strict browser execution blocks external requests and checks that the creative renderer produces visible output without script or page-load failure. Finally, the layout audit examines the selected frame set for clipping, non-permitted overlap, text occlusion, and unstable stage geometry.

When browser or layout evaluation fails, the system returns the previous assets, verified digest, and a structured report naming the offending elements, boxes, and frames. One bounded PVCr repair call must return a complete replacement of the three scoped assets while preserving the shell and verified context. If generation, extraction, sanitization, browser execution, layout audit, or repair still fails, the creative assets are discarded and the release falls back to W_V . Thus PVCr cannot make availability of the verified tutor depend on successful creative generation.

Frozen Generation and Evaluation Provenance

The reported Full-200 PVCr run starts from 200 frozen Verified Views and makes 247 model calls: 200 initial creative generations plus 47 bounded creative-quality repairs. The final set contains one Creative View per verified artifact. All 200 final views pass browser smoke and the implemented strict-layout gate over 1,494 audited frames. These are selected-final presentation outcomes; the 47 repairs remain part of the reported cost rather than being hidden as first-pass success.

Three evidence layers are kept separate. The browser/layout report measures whether the generated stage can be integrated and viewed. The five-method screenshot rubric compares visible presentation quality among frozen pages. The stricter scene-salience and algorithm-readability reports stress whether the visual design foregrounds the problem structure. None of these layers replaces the SSV result, trace, projection, or non-interference audits. The qualitative figures therefore pair the same Verified and Creative Views and distinguish direct Verified-View behavior checks from inherited artifact provenance.

Full Conditional Derivations

These paper-and-pencil results state implications under explicit representation and transition premises.

Execution-to-Trace Fidelity

Definition 1 (Instrumented execution record). For run identifier ρ , let $E_\rho = (y_\rho, T_\rho, Z_\rho, L_\rho)$ contain the answer, $T_\rho = (e_0, \dots, e_n)$, per-step service snapshots $Z_\rho = (z_0, \dots, z_{n+1})$, and source callsites $L_\rho = (\ell_0, \dots, \ell_n)$. Each service operation o_t has the atomic semantics

$$(z_t, o_t) \mapsto (z_{t+1}, e_t),$$

and e_t contains the service identity, before/after values, ρ , and runtime-captured ℓ_t .

Theorem 1 (Trace fidelity for service-mediated state). Assume (i) every canonical fact exported to the tutor is owned by a *TraceSession* service or explicit checked event; (ii) each service operation atomically performs the transition above; (iii) run identifiers and callsites are runtime-issued rather than model-authored; and (iv) the validator rejects canonical mutations outside the service boundary. Then for every $k \in \{0, \dots, n+1\}$,

$$\text{Replay}(z_0, e_{<k}) = z_k.$$

Moreover, y_ρ , T_ρ , Z_ρ , and L_ρ are observations of the same instrumented execution.

Proof. Induct on k . The empty prefix yields z_0 . If replay and execution agree at z_k , the atomic service semantics applies the same typed transition e_k to both and yields z_{k+1} . The visible-state closure premise rules out an unlogged canonical transition between steps. Runtime-issued ρ binds the record components, and each ℓ_t is captured at the operation that emits e_t . $\square$

The theorem is intentionally relative to the instrumented execution and the service-mediated state boundary. It does not imply that y_ρ or the executed transition sequence is the intended algorithm. Algorithm correctness is tested separately through oracle agreement and reference tests.

Canonical Projection and Semantic Preservation

Definition 2 (Canonical algorithmic projection). Let $\mathcal{A}$ be the representation-independent algorithm-state space containing stable object identifiers, values, marks, pointers, graph edges, stack or queue contents, the current step, and the final answer. For trace frame T_t , scene frame G_t , and reachable runtime state R_t , define

$$\alpha_T : T_t \rightarrow \mathcal{A}, \quad \alpha_G : G_t \rightarrow \mathcal{A}, \quad \alpha_R : R_t \rightarrow \mathcal{A}.$$

Layout coordinates, CSS, explanatory prose, hint visibility, prediction text, and learning-log contents are excluded. The implementation serializes these projections canonically, so equality means equality of normalized algorithm facts rather than pixel equality.

Theorem 2 (Compositional semantic preservation). For input x , let E_ρ be an accepted execution record with answer y_ρ , $T = (T_0, \dots, T_n)$ its semantic trace, $G = (G_0, \dots, G_n)$ its compiled scenes, and $R(G)$ the reachable runtime execution. Suppose

$$\begin{aligned} \text{Final}(T) &= y_\rho, \\ \alpha_T(T_t) &= \alpha_G(G_t) \quad \text{for every } t, \\ \alpha_G(G_t) &= \alpha_R(R_t) \quad \text{for reachable } t \end{aligned}$$

for every corresponding reachable frame t . Then $\alpha_T(T_t) = \alpha_R(R_t)$ at every such frame, and the rendered final answer equals y_ρ . If $y_\rho = \text{Oracle}(x)$, it also equals the oracle.

Proof. The projection statement follows by equality transitivity. At the final frame, the answer component is $\text{Final}(T) = y_\rho$; substituting oracle equality proves the final clause. $\square$

This theorem applies to the selected canonical projection; oracle agreement evaluates algorithm correctness separately.

Pedagogical Noninterference

Let runtime state be $\sigma = (a, p)$, where $a \in \mathcal{A}$ is protected algorithm state and p is pedagogical state. Let $\pi_A(a, p) = a$. For validated variant traces $\mathcal{T} = \{T^{(1)}, \dots, T^{(k)}\}$, define

$$\mathcal{V}(\mathcal{T}) = \bigcup_{j=1}^k \{\alpha_T(T_0^{(j)}), \dots, \alpha_T(T_{n_j}^{(j)})\}.$$

Theorem 3 (Pedagogical noninterference). Suppose every pure pedagogical action u satisfies $\delta_u(a, p) = (a, p')$ for some p' , and every navigation or variant action selects an algorithm state in $\mathcal{V}(\mathcal{T})$. Then a finite teaching-only sequence preserves its initial a , while every finite mixed teaching/navigation sequence keeps algorithm state in $\mathcal{V}(\mathcal{T})$.

Proof. Induct on sequence length. A pure pedagogical action preserves a . A navigation action replaces the current member of $\mathcal{V}(\mathcal{T})$ with another member, including when a variant switch changes the active trajectory. Each step preserves the corresponding invariant. $\square$

The browser stress suite operationalizes this premise by searching for violations through algorithm-state, step, and artifact hashes.

Nested Survival and Ideal Checkpoint Recovery

Proposition 1 (Nested contract survival). For sequential contract events $C_1, \dots, C_m$ with nonzero conditioning probabilities,

$$\Pr\left(\bigcap_{i=1}^m C_i\right) = \prod_{i=1}^m \Pr\left(C_i \mid \bigcap_{j=1}^{i-1} C_j\right).$$

This is the probability chain rule and requires no independence assumption. A fixed ordering localizes the first boundary at which an artifact leaves the cumulative audit sequence; causal attribution comes from the separate ablations.

Theorem 4 (Ideal stage-local recovery). Consider k sequential stages with cost $c_i > 0$ and independent, stationary per-attempt success probability $p_i \in (0, 1]$. Suppose a validator identifies the failing stage exactly; each successful stage is checkpointed; retry is independent and preserves prior stages; and verified downstream work is not recomputed. Then

$$\mathbb{E}[C_{\text{local}}] = \sum_{i=1}^k \frac{c_i}{p_i}, \quad \mathbb{E}[C_{\text{global}}] = \sum_{i=1}^k \frac{c_i}{\prod_{j=i}^k p_j},$$

and $\mathbb{E}[C_{\text{local}}] \leq \mathbb{E}[C_{\text{global}}]$.

Proof. Geometric retry yields local multiplicity $1/p_i$ and global multiplicity $1/\prod_{j=i}^k p_j$ for stage i . Summing gives both expressions, and $\prod_{j=i}^k p_j \leq p_i$ proves the inequality. $\square$

The implemented Local policy retains the specification and then rematerializes downstream stages, so its recovery experiment evaluates semantic reuse rather than the theorem’s fully checkpointed regime.

Measured Nested Contract Survival

The nested levels are: C1 visible-answer match; C2 C1 plus page load; C3 C2 plus reachable interaction; C4 C3 plus correct and wrong feedback; C5 C4 plus hint, answer reveal, and learning log; and C6 C5 plus protected-answer stability. Values in Table S13 are cumulative. Conditional survival at level i is the cumulative C_i count divided by the cumulative $C(i-1)$ count; no independence assumption is made. C6 measures protected-answer stability, while the randomized stress test evaluates full-state noninterference.

The cumulative results show that monolithic pages continue to lose surviving artifacts at load, interaction, and feedback boundaries, while factored pages that pass semantic validation retain most downstream mass.

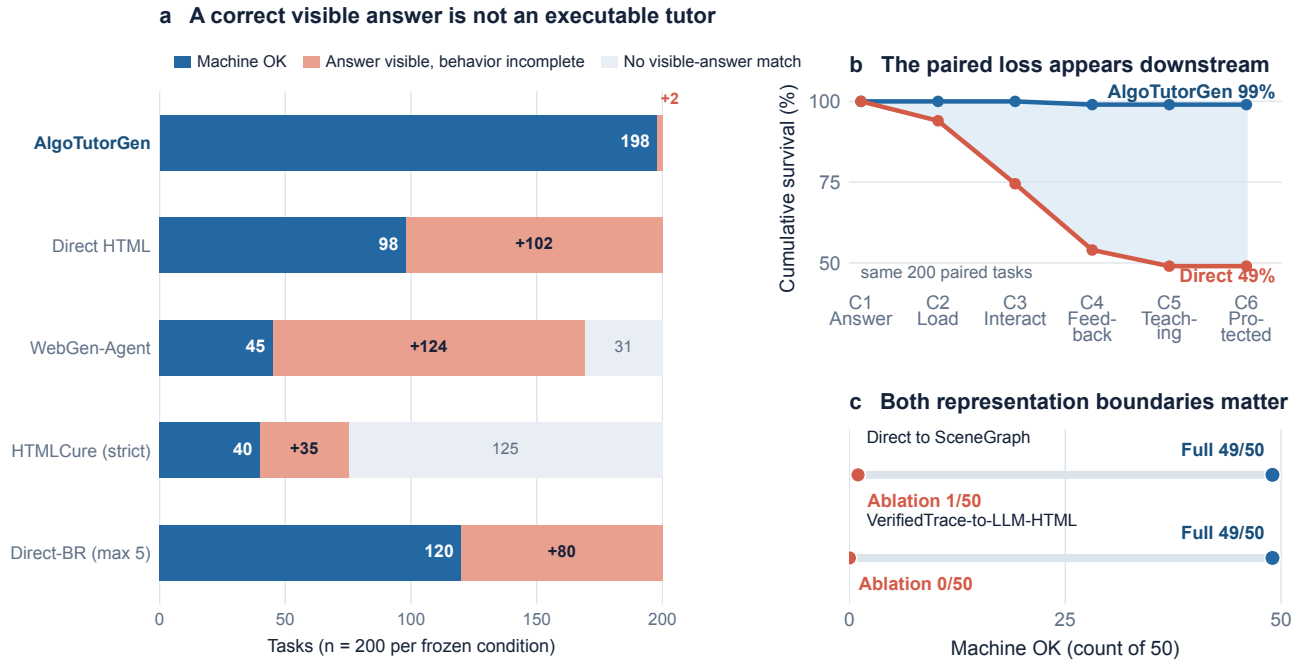


Figure S2: Full-200 reliability and representation diagnostics moved from the main paper. (a) Frozen generation conditions are partitioned into MACHINE OK, visible-answer-but-incomplete, and no-visible-answer outcomes; Direct-BR reports the adaptive best-so-far result at a maximum budget of five. Direct-BR denotes DIRECT-BROWSERREPAIR (MAX 5). (b) The primary paired ALGO TUTOR GEN–Direct comparison shows cumulative contract survival from answer visibility through protected-answer stability. (c) Two internal 50-task ablations test the executable-trace and deterministic-compilation boundaries. HTMLCURE (STRICT) blocks external resources.

Theory-Aligned Executable Audits

Representation Preservation and Determinism

The preservation audit compares the normalized algorithm-state projection stored in each trace event, scene frame, and reachable runtime frame. The projection includes stable identifiers, values, marks, pointers, graph edges, container contents, current step, and final answer; layout, CSS, and teaching text are excluded.

Set	Artifacts/pages	Frames/sequences	Primary use	Provenance boundary
Main benchmark	200	9,421 frames	Reliability, preservation, mutation	One sample per task across 23 families; selected-final artifacts
Held-out generation	39 valid of 40	–	Frozen-task generalization	Original result remains 39/40
Completed held-out audit	40	4,568 frames	Semantic/state audits	Fortieth artifact is a documented targeted retry
Long-trace	54	41,119 frames	Preservation and scalability	18 tasks at small, medium, and large scales
Noninterference	240 pages	24,000 sequences	Browser state isolation	Main 200 plus completed held-out 40

Table S12: Artifact sets used by the main and theory-aligned evaluations. The primary preservation row reuses the same selected-final pages as the main evaluation; the held-out and long-trace rows use the same instrumentation, compiler, and runtime. The completed held-out audit set contains one documented targeted retry that is not counted as an original generation pass.

Method	C1	C2	C3	C4	C5	C6
ALGO _{TUTOR} GEN main	100.0	100.0	100.0	99.0	99.0	99.0
Direct HTML main	100.0	94.0	74.5	54.0	49.0	49.0
ALGO _{TUTOR} GEN Flash final	100.0	100.0	100.0	100.0	100.0	100.0
Direct Flash	99.0	95.0	81.0	65.5	59.0	59.0
ALGO _{TUTOR} GEN GLM final	99.0	99.0	99.0	98.0	98.0	98.0
Direct GLM	100.0	91.0	53.0	38.0	17.5	17.5
ALGO _{TUTOR} GEN Kimi final	97.5	97.5	97.5	97.0	97.0	97.0
Direct Kimi	99.5	95.0	68.0	52.5	43.5	43.5

Table S13: Cumulative nested-contract survival (percent). Cross-model ALGO_{TUTOR}GEN rows use final-quality artifacts with recorded retries. The main paired comparison uses one frozen selected-final page per task; token-matched reliability appears in Table S23.

Set	Artifact pass	Equivalent frames
Main	200/200	9,421/9,421
Completed held-out	40/40	4,568/4,568
Long-trace	54/54	41,119/41,119
Total	294/294	55,108/55,108

Table S14: Trace–scene–runtime projection audit.

Twenty stratified artifacts were recompiled and rerendered ten times each. Every artifact produced one canonical projection hash and one render hash across reruns. These observations establish representation-level agreement and tested-environment stability; independent oracle and human audits evaluate source-to-trace validity.

Semantic Mutation and Overlay Isolation

The mutation suite distinguishes changes that must be rejected from transformations that should be accepted. Ambiguous event deletions are not automatically classified as semantic faults because a deleted explanation or repeated mark may be redundant. Reordered unordered-set results are compared with a case-aware equivalence relation.

Class	Expected	Observed
Semantic violations	Reject	2,198/2,198
Teaching-text rewrites	Accept	195/195
Visual-metadata changes	Accept	195/195
Equivalent reorderings	Accept	2/2
Preserving total	Accept	392/392

Table S15: Contract discrimination on the defined mutation suite.

Condition	Outcome
Four legal overlay variants	372/372 each
Illegal <code>final_answer/state</code> fields	372/372 sanitized
Negative step	372/372 rejected
Nonexistent step	372/372 warned
Cross-model GLM overlay	369/369

Table S16: Teaching-overlay isolation. Outcomes preserve the canonical algorithm-state hash unless marked as rejected or warned.

The legal variants are original, concise, detailed, and random text overlays; their text changes do not redefine algorithm state. The existing sanitizer removes the tested protected writes, the schema rejects negative indices, and step-not-found warnings prevent unbound teaching steps. For the GLM overlay, 169 cases apply fully and 200 apply partially because model-specific traces expose different step sets. The illegal-field result is specifically sanitization rather

than schema-level `extra=forbid`. Together, these tests measure discrimination across the defined mutation and preserving suites.

Pedagogical Noninterference Stress Test

Quantity	Count
Unique pages	240
Random action sequences	24,000
Total actions	1,561,298
Pure pedagogical actions	435,859
Navigation/variant actions	1,125,439
Pages passed	240/240
Overlay artifacts passed	240/240
Observed violations	0

Table S17: Browser noninterference stress-test coverage.

Pure actions are correct submission, wrong submission, hint, show answer, and clear learning log. Navigation actions are next, previous, timeline selection, reset, and variant selection. Each page receives 100 random sequences of 30–100 actions. Pure actions must preserve artifact, state, and step hashes; navigation must preserve the artifact and select the expected verified frame. No sequence produces a noninterference counterexample under this action grammar.

Local Resume Versus Global Restart

Both policies use the same structured output space, validators, and maximum of three policy decisions. Local keeps the current solution specification, repairs it, then reruns materialization and teaching. Global discards the specification and independently regenerates it before rerunning the same downstream work.

A. End-to-end outcomes

Model	Policy	Succ.	Calls	Tokens	Time	p
Flash	LOCAL RESUME	38/50	6.63	71,369	172.9 s	0.4545
Flash	GLOBAL RESTART	42/50	5.50	62,256	194.2 s	
GLM	LOCAL RESUME	42/50	6.69	92,385	533.8 s	1.0000
GLM	GLOBAL RESTART	43/50	6.65	96,186	558.2 s	

B. Paired success counts

Model	Both	Local only	Global only	Neither
Flash	32	6	10	2
GLM	37	5	6	2

C. Token decomposition per successful page

Model	Policy	Spec/repair	Teaching	Total
Flash	LOCAL RESUME	25,031	46,338	71,369
Flash	GLOBAL RESTART	25,781	36,476	62,256
GLM	LOCAL RESUME	37,342	55,043	92,385
GLM	GLOBAL RESTART	47,462	48,724	96,186

Table S18: Recovery outcomes, paired success counts, and token decomposition. Calls and tokens are per successful page and include final-failure costs; time is time-to-valid on successful cases.

Flash favors Global numerically in success and tokens per success. GLM gives Local slightly lower token and time cost but one fewer success. Neither paired success difference is

significant under the current rematerializing implementation, which differs from the theorem’s fully checkpointed regime.

Additional Reliability Evidence

Held-Out Tasks and Full-Sample Replay

Set	ALGO TUTOR GEN	Direct
Held-out generation	39/40	40/40
Held-out MACHINE OK	39/40	18/40
Held-out self-contained	40/40	40/40

Table S19: Additional task/input robustness.

The held-out set contains 40 new cases from 15 families. The MACHINE OK difference is +52.5 percentage points (95% paired-bootstrap CI [37.5, 67.5], exact McNemar $p = 9.54 \times 10^{-7}$). The targeted retry used to complete the semantic audit does not retroactively change 39/40 generation.

Non-Degenerate Ablations and Browser Repair

Condition	Full	Ablation
Direct-to-SceneGraph	49/50	1/50
VERIFIED TRACE-TO-LLM-HTML	49/50	0/50

Table S20: Non-degenerate 50-task ablations. Both outputs are self-contained under the corrected resource parser.

Direct-to-SceneGraph isolates the fixed runtime without an executable trace and validation; its 96-point difference has a 95% CI of [90, 100] and exact McNemar $p = 7.11 \times 10^{-15}$. VERIFIED TRACE-TO-LLM-HTML isolates unconstrained page generation after a verified trace: its visible answer is 50/50, but interaction reachability and MACHINE OK are 0/50. On these 50-task ablations, neither a fixed renderer nor a verified trace followed by unconstrained page generation reproduces the full-chain reliability.

All 200 tasks reuse the same hash-verified frozen initial pages; none is regenerated. Each failed page is audited in Playwright, and the model receives generic browser observations plus the complete previous HTML, but not the hidden nine-field audit outcomes. The policy stops immediately after MACHINE OK and otherwise selects the historical page with the most passing checks, breaking ties toward the earlier page. The budget sweep therefore evaluates nested adaptive policies rather than forced final rewrites.

The pass curve is $106 \rightarrow 118 \rightarrow 119 \rightarrow 120 \rightarrow 120$. Relative to budget zero, exact paired McNemar p -values are 0.000488, 0.000244, and 0.000122 for budgets one, two, and three/five, respectively; these are cumulative within-task comparisons, not independent samples. The Wilson 95% interval moves from [46.1, 59.8]% at budget zero to [53.1, 66.5]% at budgets three and five. The first repair recovers 12 tasks, the second and third recover one each, and the fourth and fifth recover none. Across 417 repair calls, page-level fail-to-pass occurs 14 times and pass-to-fail zero times because successful pages stop.

The 417 repairs use 15,647,489 tokens (6,998,333 input and 8,649,156 output). The concurrent sweep completes in

Max repairs	MACHINE OK	Δ from start	Repair calls	Repairs/task	Repair tokens/task	Cum. latency/task
0	106/200 (53.0%)	0	0	0.000	0	0.0 s
1	118/200 (59.0%)	+12	94	0.470	16,405	94.1 s
2	119/200 (59.5%)	+13	176	0.880	31,379	177.1 s
3	120/200 (60.0%)	+14	257	1.285	46,650	259.1 s
5	120/200 (60.0%)	+14	417	2.085	78,237	424.8 s

Table S21: DIRECT-BROWSERREPAIR (MAX 5) budget sweep. A budget of r permits at most r repairs beyond the frozen initial page. Each task stops on MACHINE OK and otherwise publishes the best-so-far page. Tokens include only new repair calls. The latency column is the mean cumulative API repair latency per task, averaged over all 200 tasks.

Budget	Load	Answer	Interact	Correct FB	Wrong FB	Hint	Show	Log	Protected	MACHINE OK
0	186/200	200/200	155/200	128/200	133/200	137/200	138/200	143/200	155/200	106/200
1	200/200	200/200	165/200	139/200	144/200	151/200	151/200	157/200	165/200	118/200
2	200/200	200/200	166/200	140/200	145/200	152/200	153/200	158/200	166/200	119/200
3	200/200	200/200	166/200	140/200	146/200	152/200	153/200	158/200	166/200	120/200
5	200/200	200/200	166/200	140/200	146/200	152/200	153/200	158/200	166/200	120/200

Table S22: Nine-check outcomes under each maximum repair budget. FB denotes feedback. “Protected” is the browser audit’s protected-state check, not the stronger randomized noninterference suite.

3,714 seconds of elapsed wall-clock time. All prompts contain the complete previous HTML. These data show a real but saturating benefit from whole-page browser repair under this frozen policy.

All reliability, qualitative, VLM, and human-calibration results reported for DIRECT-BROWSERREPAIR (MAX 5) use the final best-so-far pages selected by the same adaptive max-five policy.

Qualitative Results and Case Studies

The figures in this section use a separate 23-case English qualitative package. Each case contains a saved SSV Verified View with its page, screenshot, audit, and structured artifact JSON, together with frozen comparison pages. For the fixed-shell evidence, we re-export the available stage-only PVCR asset against the corresponding saved English artifact and capture both views from one browser page at one frame. The Verified capture disables only the central-stage renderer, whereas the Creative capture restores it. The capture audit checks the same artifact and frame, equality of the normalized shell DOM outside the stage, a changed stage DOM, no page errors, and no external requests. Figure S4 shows the fixed-shell comparison in detail, while Figure S5 separately compares one Verified View with four baselines.

Fixed-shell PVCR changes only the central stage

Four English cases use the same verified artifact and frozen frame within each column.

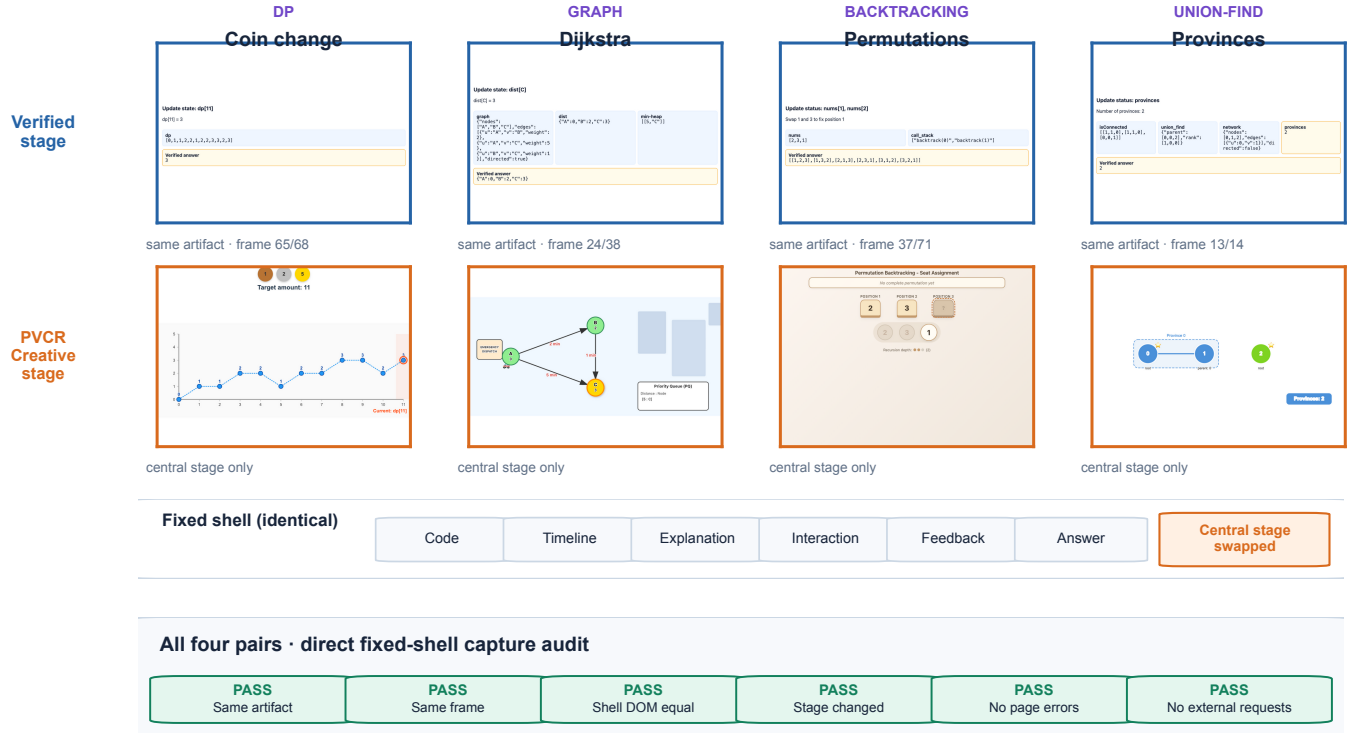
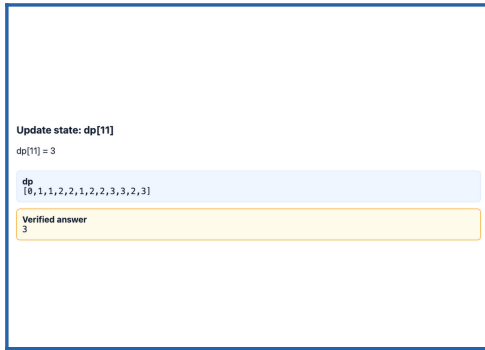
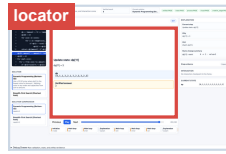


Figure S3: Same-artifact, same-frame central-stage crops for dynamic programming, shortest path, backtracking, and union–find. Within each column, the Verified View uses the deterministic fallback and the Creative View uses the stage-only PVCR renderer. The fixed-shell capture audit verifies the same artifact, same frame, identical normalized shell DOM, a changed central stage, no page errors, and no external requests for all four pairs.

One verified artifact, two fixed-shell views

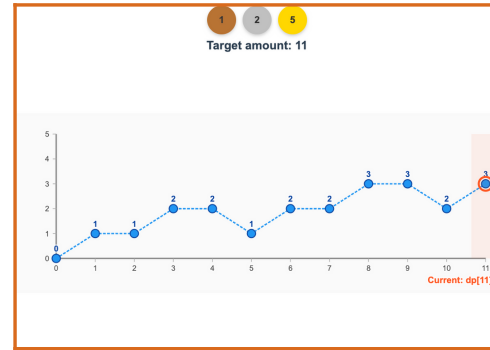
Coin Change · frame 65/68 · red locator boxes identify the central stage in the full page.

Verified stage



Deterministic fallback

PVCR Creative stage



Optional stage-only renderer

Capture audit · normalized shell SHA-256 matches

PASS Same artifact	PASS Same frame	PASS Shell DOM equal	PASS Stage changed	PASS No page errors	PASS No external requests
-----------------------	--------------------	-------------------------	-----------------------	------------------------	------------------------------

artifact 2e394a7ba0cc... · shell DOM 043b3d6e3268... · answer 3

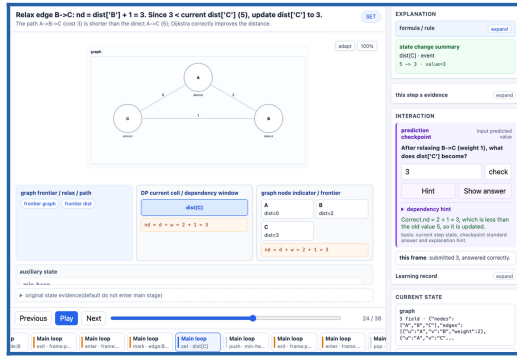
Figure S4: Detailed fixed-shell PVCR pair for the English Coin Change artifact at frame 65/68. Red locator boxes identify the central stage within the two full-page captures; the enlarged crops show the deterministic Verified stage and the stage-only PVCR rendering. The capture audit verifies the same artifact and frame, identical normalized shell DOM outside the stage, a changed stage, no page errors, and no external requests.

Same answer, different tutor behavior

Frozen Dijkstra task · graph and answer fixed · screenshots show the audited behavior boundary.

a AlgoTutorGen · learner loop

9/9 MACHINE OK



Frame 24/38 · predict dist[C] = 3 · submit · receive correct feedback

State, checkpoint, feedback, Hint/Answer controls and learning record remain on one page.

b Direct HTML · 2/9

Visible before learner action



Expected answer is already exposed

Static visibility is not evidence of a reachable teaching loop.

Audit: interaction + feedback chain FAIL

c Direct-BR (max 5) · 7/9

Checkpoint reached; partial behavior



Wrong feedback + learning log: PASS

Correct feedback + Show answer: FAIL

d Five methods x nine browser contracts

Screenshot claims map directly to the colored cells below.

		all five pass	Page load	Visible answer	Interaction	Correct feedback	Wrong feedback	Hint	Show answer	Learning log	Protected answer
AlgoTutorGen	9/9	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS	PASS
Direct HTML	2/9	PASS	PASS	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
WebGen-Agent	2/9	PASS	PASS	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
HTMLCure (strict)	2/9	PASS	PASS	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL	FAIL
Direct-BR (max 5)	7/9	PASS	PASS	PASS	FAIL	PASS	PASS	PASS	FAIL	PASS	PASS

Figure S5: A frozen Dijkstra case with a visible answer but different tutor behavior. (a) ALGO TUTOR GEN frame 24/38 shows a submitted prediction and correct feedback on the same page as the verified state and learning record. (b) DIRECT HTML exposes the expected answer before learner action, but static visibility does not establish the audited teaching loop. (c) DIRECT-BROWSER REPAIR (MAX 5) reaches a checkpoint and records wrong feedback, while its correct-feedback and answer-reveal checks fail. (d) All five methods expose the expected answer, but only ALGO TUTOR GEN satisfies all nine contracts; DIRECT HTML, WEB GEN-AGENT, and HTMLCURE (STRICT) pass 2/9, whereas DIRECT-BROWSER REPAIR (MAX 5) passes 7/9.

Model-Token Cost and Reliability

At the operational all-attempt point, ALGO_{TUTOR}GEN uses 84,352.785 tokens per task and passes 198/200 pages, while the closest realized-cost DIRECT-BROWSERREPAIR (MAX 5) replay uses 84,254.445 tokens per task and passes 120/200. The paired difference is +39.0 percentage points (95% paired-bootstrap CI [32.0, 46.0], exact McNemar $p = 1.41 \times 10^{-21}$). Matching the selected-final cost gives the same 198/200 versus 120/200 result, and the maximum observed DIRECT-BROWSERREPAIR (MAX 5) policy remains at 120/200 while using 97,914.885 tokens per task. Thus, under the frozen whole-page repair policy, matching model-token cost does not close the observed reliability gap; this comparison does not isolate the causal contribution of individual system components.

Condition	Calls/task	Tokens/task	MACHINE OK
DIRECT-BROWSERREPAIR (MAX 5), no repair	1.000	19,677	106/200
ALGO _{TUTOR} GEN selected-final	5.330	76,847	198/200
DIRECT-BROWSERREPAIR (MAX 5), selected-final matched	2.575	76,854	120/200
ALGO _{TUTOR} GEN operational all-attempt	5.755	84,353	198/200
DIRECT-BROWSERREPAIR (MAX 5), all-attempt matched	2.760	84,254	120/200
DIRECT-BROWSERREPAIR (MAX 5), maximum observed	3.085	97,915	120/200

Table S23: Total model-token cost and reliability. Direct-repair totals include the frozen initial page and all included repair calls. No stable price configuration was frozen, so monetary cost is not reported.

Long-Trace Scalability

All 54 artifacts materialize, while 52 complete browser measurement. The exporter currently embeds complete frames in a self-contained document, so repeated state dominates size and load cost.

Scale	Frames	HTML	Load	Step	Peak heap
Small	104.4	1.40 MB	120 ms	14.7 ms	6.3 MB
Medium	543.3	23.25 MB	1,933 ms	45.4 ms	59.1 MB
Large	1,636.7	160.42 MB	8,354 ms	101.3 ms	185.6 MB

Table S24: Long-trace browser costs; all entries are means.

The large KMP page contains 3,063 frames and 581 MB of HTML. The large sliding-window-unique page contains 5,533 frames and 1.08 GB of HTML. Both exceed the 60-second load budget. These results motivate event deltas, frame virtualization, lazy loading, or state compression when scaling beyond the evaluated regime.

Failure Analysis

Selected-Final Browser Failures

The main ALGO_{TUTOR}GEN condition generates valid artifacts for all 200 tasks, but two pages fail the final conjunction. `stack_valid_parentheses_full_core` fails wrong-answer feedback; `string_longest_common_`

`prefix_full_core` fails both correct- and wrong-answer feedback. Both pages pass load, visible answer, reachable interaction, hint, answer reveal, learning log, and protected-answer stability. The first failing boundary is therefore the teaching-feedback contract, not solver materialization or scene compilation.

The final cross-model failures separate missing artifacts from generated pages with incomplete feedback. Flash has no final failure. GLM fails to generate valid artifacts for two tasks and has two generated pages with bidirectional-feedback failure. Kimi fails to generate five artifacts and has one generated page with bidirectional-feedback failure. The held-out failure is a materialization/generation failure; its missing page consequently fails all downstream browser checks. These categories are not merged into one generic “page failure.”

Baseline Failure Mechanisms

Direct HTML has 12 load failures caused by JavaScript syntax, duplicate declarations, invalid escapes, or null-DOM operations. Among accepted HTMLCURE (STRICT) rewrites, 125 introduce a Google Fonts request and therefore fail the strict self-contained condition when external requests are blocked. WebGen-Agent has one rendering timeout. The Dijkstra case in Figure S5 was selected because every page loads and shows the answer, localizing the contrast to interaction and teaching behavior rather than a trivial parse failure.

Secondary Visual Evaluation

The formal visual comparison uses the 200 frozen selected-final pages for each of five methods. The ALGO_{TUTOR}GEN column is explicitly the PVC_R Creative View generated from 200 SSV Verified Views rather than the deterministic Verified View. Scores come from the same screenshot rubric and the frozen output/experiments/all_method_auxiliary_eval_20260718/all_method_auxiliary_eval_report.json report summarized in the detailed result ledger. One WebGen-Agent render failure is assigned the minimum score; the other 999 pages are VLM-scored. Correctness and release use the separate executable audits.

Method	Overall	Alignment	State	Process	Instruction	All ≥ 3
PVCR	4.755	4.640	4.670	4.915	4.795	199/200
Direct	4.561	4.455	4.605	4.660	4.525	186/200
WebGen	4.441	4.500	4.645	4.150	4.470	177/200
HTMLCURE (STRICT)	4.300	4.350	4.445	4.050	4.355	163/200
DIRECT-BROWSERREPAIR (MAX 5)	4.670	4.590	4.725	4.730	4.635	191/200

Table S25: Five-method visual evaluation under one frozen screenshot rubric. All scores are 1–5 means. “All ≥ 3 ” counts pages meeting the threshold on all four dimensions. These are single-VLM proxy ratings, not human judgments.

PVCR is highest on the overall, alignment, process, and instructional-design means. The following human subset independently calibrates this single-VLM proxy.

Separate PVCR-only reports audit the same 200 final Creative Views rather than comparing methods. All 200 pass browser smoke and strict layout, covering 1,494 frames with zero recorded non-permitted overlaps, clipping events, or text occlusions. A stricter scene-salience stress rubric identifies 51/200 high-salience pages, while algorithm readability passes 193/200; this diagnostic targets visual emphasis beyond the release gate. SSV correctness is evaluated separately.

Human Calibration of Visual Evaluation

Two reviewers from the research team independently evaluated a stratified subset of 30 tasks and five methods, yielding 150 anonymized pages. Method identities and the other reviewer’s ratings were hidden until both reviewers completed scoring. Each page received a 1–5 score for problem–visual alignment, algorithm-state readability, process-change clarity, and instructional visual design. Page-level human scores are the mean of the two reviewers. The evaluated ALGO-TUTORGEN condition is the PVCR Creative View; the four baselines are Direct HTML, WebGen-Agent, HTMLCURE (STRICT), and DIRECT-BROWSERREPAIR (MAX 5).

Method	Align.	State	Process	Instruction	Overall	All ≥ 3
PVCR Creative View	4.39	4.33	4.27	4.38	4.34	29/30
Direct HTML	3.38	3.22	3.06	3.18	3.21	21/30
WebGen-Agent	3.67	3.58	3.47	3.61	3.58	24/30
HTMLCURE (STRICT)	3.54	3.42	3.29	3.39	3.41	23/30
DIRECT-BROWSERREPAIR (MAX 5)	3.81	3.69	3.57	3.72	3.70	26/30

Table S26: Human visual ratings on 30 tasks per method. “All ≥ 3 ” requires the averaged human score to reach 3 on every dimension.

Table S26 shows that PVCR has the highest human mean in all four dimensions and Overall. Its Overall advantage over the strongest baseline, DIRECT-BROWSERREPAIR (MAX 5), is 0.64 points, and 29/30 Creative Views meet the four-dimension threshold. Across methods, 123/150 pages meet this human threshold.

Inter-reviewer reliability is summarized using quadratic weighted kappa (QWK).

Dimension	Human–VLM ρ	QWK	Within 1
Alignment	0.72	0.76	144/150 (96.0%)
State readability	0.67	0.72	142/150 (94.7%)
Process clarity	0.62	0.66	139/150 (92.7%)
Instructional design	0.65	0.70	141/150 (94.0%)
Overall	0.73	–	–

Table S27: Human–VLM rank correlation and inter-reviewer reliability. Weighted κ is quadratic; the last column counts reviewer scores differing by at most one point. Reviewer A/B Spearman correlations for the four dimensions are 0.73, 0.69, 0.64, and 0.68, respectively.

The averaged human Overall score correlates with VLM Overall at Spearman $\rho = 0.73$; dimension-level correlations range from 0.62 to 0.72. Inter-reviewer quadratic weighted κ ranges from 0.66 to 0.76, and 92.7–96.0% of paired ratings

differ by no more than one point. Exact reviewer agreement is 106/150 for alignment, 101/150 for state readability, 95/150 for process clarity, and 99/150 for instructional design.

For the binary decision that all four dimensions reach 3, human and VLM judgments agree on 135/150 pages (90.0%; Cohen’s $\kappa = 0.656$). The confusion counts are 116 pages accepted by both, seven accepted only by humans, eight accepted only by the VLM, and 19 rejected by both. Thus humans accept 123 pages and the VLM accepts 124. Disagreements mainly involve attractive pages with weak algorithm-state detail, static screenshots that obscure transitions, dense but still usable layouts, or different emphasis on local labels versus overall teaching clarity.

Baseline	Ours better	Base better	Tie	Mean Δ
Direct HTML	27/30	1/30	2/30	+1.13
WebGen-Agent	24/30	3/30	3/30	+0.76
HTMLCURE (STRICT)	26/30	2/30	2/30	+0.93
DIRECT-BROWSERREPAIR (MAX 5)	22/30	4/30	4/30	+0.64

Table S28: Paired task-level comparison using averaged human Overall scores. Human and VLM method-level winner directions agree for all four baselines.

The calibration met the criteria specified before scoring: Overall Spearman is at least 0.60; all four dimension correlations exceed 0.50 and are positive; binary threshold agreement exceeds 80%; human and VLM winner directions agree for four of four baselines; each dimension has weighted $\kappa \geq 0.60$; and more than 90% of reviewer pairs differ by at most one point. These results support the VLM score as a scalable approximation of human visual judgment on this frozen subset. Student learning outcomes are outside the present evaluation.

Reproducibility

The accompanying anonymous Code and Data Supplement includes the implementation, benchmark definitions, exact production prompts, referenced JSON reports, qualitative artifacts, and a SHA-256 manifest. The principal entry points within that artifact are:

- docs/EXPERIMENT_RESULTS_DETAILED.md, which records the final reporting hierarchy, configurations, totals, and source-report paths;
- algolab/generation/prompts/ and the dynamic prompt builders in algolab/generation/;
- artifacts/method_comparison_samples_en/manifest.json, covering 23 cases, Verified-View artifacts, frozen comparison pages, screenshots, and audit records; and
- latex/figure-generation/AlgoTutorGen-ssv-pvcr-artifact-showcase-editable.pptx and latex/figure-generation/figure2-system-architecture-ssv-pvcr-editable.pptx, which provide the editable qualitative and

system-architecture compositions aligned with the formal SSV/PVCR terminology.

Browser evaluation uses a 1365×900 viewport, waits 500 ms after DOM content load, searches at most 80 timeline steps for a checkpoint, and blocks external HTTP requests in strict conditions. Paired bootstrap uses seed 20260713 with 10,000 draws; noninterference sequences use seed 20260714. The recorded browser build is Playwright Chromium 1223. Remote model versions can drift, and self-contained artifacts cannot reproduce a retired remote service exactly.

References

Authors, A.; Ma, Y.; Lin, Y.; Zeng, H.; Zheng, X.; and Ji, R. 2026. ALGOGEN: Tool-Generated Verifiable Traces for Reliable Algorithm Visualization. In Liakata, M.; Moreira, V. P.; Zhang, J.; and Jurgens, D., eds., *Findings of the Association for Computational Linguistics: ACL 2026*, 3168–3189. San Diego, California, United States: Association for Computational Linguistics.